Day 102 — Line-by-Line Syntax Explanation

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 15, Day 5 reference

Watch: Applying BERT to Question Answering (SQuAD v1.1) (InnerWorkingsAI) | Text Extraction From a Corpus Using BERT (AKA Question Answering) (Abhishek Thakur)

`day102_question_answering.py`, segment by segment, in the order `main()` uses things. Day 99's bootstrap and Day 101's sentence splitter are imported; the span search, metrics and baseline are from scratch. Code boxes are cut from the real script by program (inline comments removed). Purple **MATH** boxes come before code that implements a formula. A glossary is at the end.

MATH BOX LEGEND

Symbols: s_i, e_j = start and end scores of tokens i and j · EM = exact match · F1 = word-overlap F1 · AUC = area under the ROC curve

Segment 0 — Setup

Imports and settings

CODE

```
HERE = os.path.dirname(os.path.abspath(__file__))
sys.path.insert(0, HERE)
with contextlib.redirect_stdout(io.StringIO()):
    import day99_exercise as d99
    import day101_summarization as d101
if hasattr(sys.stdout, "reconfigure"):
    sys.stdout.reconfigure(encoding="utf-8")
hf_logging.set_verbosity_error(); hf_logging.disable_progress_bar()
hub_logging.set_verbosity_error(); disable_progress_bars()
torch.set_num_threads(4)
MODEL = "distilbert/distilbert-base-cased-distilled-squad"
N_Q = 1000
MAX_LEN, STRIDE, MAX_ANSWER = 384, 128, 30
```

- `import day101_summarization as d101` — importing a lesson script for one function: its main() does not run.
- `MAX_LEN, STRIDE, MAX_ANSWER = 384, 128, 30` — window length, overlap between windows, longest answer, all in tokens.

Segment 1 — Answering

answer: the best span over every window

MATH

$$(i^*, j^*) = \operatorname{argmax} s_i + e_j \quad \text{subject to } i \le j < i + 30, \text{ both in the paragraph}$$

CODE

```
def answer(tok, model, question, context):
    enc = tok(question, context, truncation="only_second", max_length=MAX_LEN, stride=STRIDE,
              return_overflowing_tokens=True, return_offsets_mapping=True, padding=True,
                  return_tensors="pt")
    offsets = enc.pop("offset_mapping").numpy(); enc.pop("overflow_to_sample_mapping")
    with torch.no_grad():
        out = model(**enc)
    best = (-np.inf, "")
    for w in range(len(offsets)):
        seq = enc.sequence_ids(w)
        ctx = np.array([s == 1 for s in seq])
        st, en = out.start_logits[w].numpy(), out.end_logits[w].numpy()
        S = st[:, None] + en[None, :]
        ok = np.triu(np.ones_like(S, dtype=bool)) & ~np.triu(np.ones_like(S, dtype=bool),
            MAX_ANSWER)
        ok &= ctx[:, None] & ctx[None, :]
        S = np.where(ok, S, -np.inf)
        i, j = np.unravel_index(np.argmax(S), S.shape)
        if S[i, j] > best[0]:
            best = (S[i, j], context[offsets[w][i][0]:offsets[w][j][1]])
    return best[1], float(best[0])
```

- `truncation="only_second", stride=STRIDE, return_overflowing_tokens=True` — cut only the paragraph, into overlapping windows, each with the full question.
- `return_offsets_mapping=True` — for every token, its (start, end) character positions in the original text.
- `enc.sequence_ids(w)` — 0 for question tokens, 1 for paragraph tokens, None for special tokens.
- `st[:, None] + en[None, :]` — broadcasting: a (tokens, tokens) table of every start + end score.
- `np.triu(...) & ~np.triu(..., MAX_ANSWER)` — upper triangle: end >= start; minus the band 30 or more above: end < start + 30.
- `np.unravel_index(np.argmax(S), S.shape)` — the (row, column) of the largest entry of a 2-D table.

Segment 2 — Metrics and baseline

normalize, exact_match and f1: SQuAD's rules

MATH

$$F1 = 2PR / (P + R), \quad P = \text{shared / predicted words}, \quad R = \text{shared / gold words (after normalising)}$$

CODE

```
def normalize(s):
    s = "".join(ch for ch in s.lower() if ch not in set(string.punctuation))
    s = re.sub(r"\b(a|an|the)\b", " ", s)
    return " ".join(s.split())
def exact_match(pred, golds):
    return float(any(normalize(pred) == normalize(g) for g in golds))
def f1(pred, golds):
    def one(p, g):
        p, g = normalize(p).split(), normalize(g).split()
        common = sum((Counter(p) & Counter(g)).values())
        if common == 0:
            return 0.0
        prec, rec = common / len(p), common / len(g)
        return 2 * prec * rec / (prec + rec)
    return max(one(pred, g) for g in golds)
```

- `string.punctuation` — a ready-made string of ASCII punctuation characters.
- `re.sub(r"\b(a|an|the)\b", " ", s)` — remove the articles as whole words only.
- `Counter(p) & Counter(g)` — & on Counters keeps the smaller count of each word: the shared words, with repeats.
- `max(one(pred, g) for g in golds)` — the best score against any of the acceptable answers.

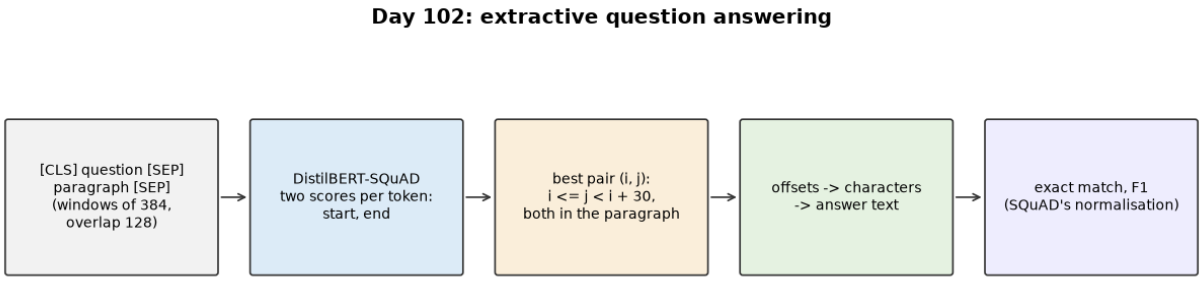
overlap_sentence: the baseline

CODE

```
def overlap_sentence(question, context):
    stop = {"the", "a", "an", "of", "in", "to", "is", "was", "what", "which", "who", "when",
        "where", "how", "why", "did", "does",
            "do", "and", "for", "on", "by", "with", "as", "at", "that", "from", "are", "were",
                "it", "its"}
    q = {w for w in re.findall(r"\w+", question.lower()) if w not in stop}
    sents = d101.split_sentences(context) or [context]
    return max(sents, key=lambda s: len(q & set(re.findall(r"\w+", s.lower()))))
```

- `sents = d101.split_sentences(context) or [context]` — Day 101's splitter; 'or' falls back to the whole paragraph if it returns nothing.
- `max(sents, key=lambda s: ...)` — the sentence with the largest shared-word count.

SYSTEM DIAGRAM



Extractive question answering, step by step.

Segment 3 — The wrong paragraph

demo_wrong_paragraph: can the score detect it?

MATH

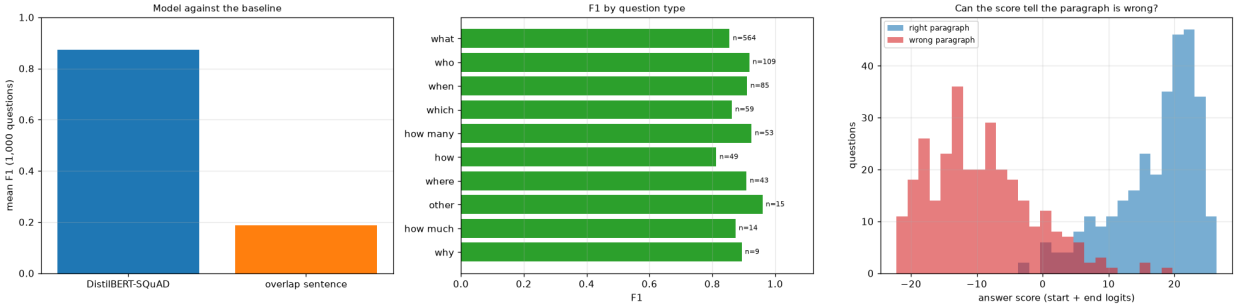
$$AUC = \int TPR \, d(FPR) = \text{the chance a right-paragraph score beats a wrong-paragraph score}$$

CODE

```
titles = [q["title"] for q in Q]
wrong = []
for i, q in enumerate(Q[:300]):
    j = next(k for k in range(i + 1, i + len(Q)) if titles[k % len(Q)] != titles[i])
    wrong.append(answer(tok, model, q["question"], Q[j % len(Q)]["context"]))
right_s = np.array([s for _, s in preds[:300]]); wrong_s = np.array([s for _, s in wrong])
print(f"\nscore (start + end) with the right paragraph: median {np.median(right_s):.2f}; with a
    wrong one: median {np.median(wrong_s):.2f}")
print("sample of answers from the wrong paragraph: " + "; ".join(f"{q['question'][:40]!r} ->
    {t[:30]!r}" for q, (t, _) in list(zip(Q, wrong))[:3]))
pos = np.r_[right_s, wrong_s]; lab = np.r_[np.ones(300), np.zeros(300)]
order = np.argsort(-pos); tp = np.cumsum(lab[order]); fp = np.cumsum(1 - lab[order])
auc = np.trapezoid(np.r_[0, tp / 300], np.r_[0, fp / 300])
```

- `next(k for k in range(...) if titles[k % len(Q)] != titles[i])` — the first later question from a different article; % wraps around the list.
- `np.argsort(-pos)` — all 600 scores from highest to lowest: the order a threshold would sweep through.
- `np.cumsum(lab[order])` — true positives so far, as the threshold drops past each score.
- `np.trapezoid(y, x)` — the area under a curve by the trapezoid rule: here the ROC curve's AUC.

REAL GRAPH



Real graph from the script: the model against the baseline, F1 by question type, and answer scores with the right and the wrong paragraph.

Quick Syntax Glossary — new or reinforced constructs

- `return_overflowing_tokens=True, stride=128` — long inputs become several overlapping windows.
- `return_offsets_mapping=True` — character positions of every token, to cut the answer out of the original text.
- `enc.sequence_ids(i)` — which part of a question-paragraph pair each token belongs to.
- `np.unravel_index(k, shape)` — a flat index back to (row, column).
- `Counter(a) & Counter(b)` — multiset intersection: shared items with their smaller counts.
- `np.trapezoid(y, x)` — numerical integration by trapezoids (NumPy 2's name for trapz).

End of Day 102 syntax reference. For the experiments and what they showed, see the Day 102 complete guide.